

CS 3224

Operating Systems

Professor G. Sandoval

01

About me

- 20 yrs at Microsoft
- 10 years at NYU
- Research Deep Learning and Security



Why are you here? Why study OS?

- **Concurrency:** threads, locks, deadlocks, race conditions.
- **Resource Management**
- **Perf Analysis and Contention Resolution:** queuing, CPU Scheduling, prioritization
- **Interfaces and Hiding Complexity:** sockets, file systems, address spaces.
- **Dealing with Big Software**
- **Getting a Job!!**

Course Goals

- Understand **operating system design** and implementation
- Confidence with the **C programming** Language
- **Assembly language** Programming



Administrivia

How to unblock : EdStem

- EdStem
- We will be linking with Brightspace shortly

Homework

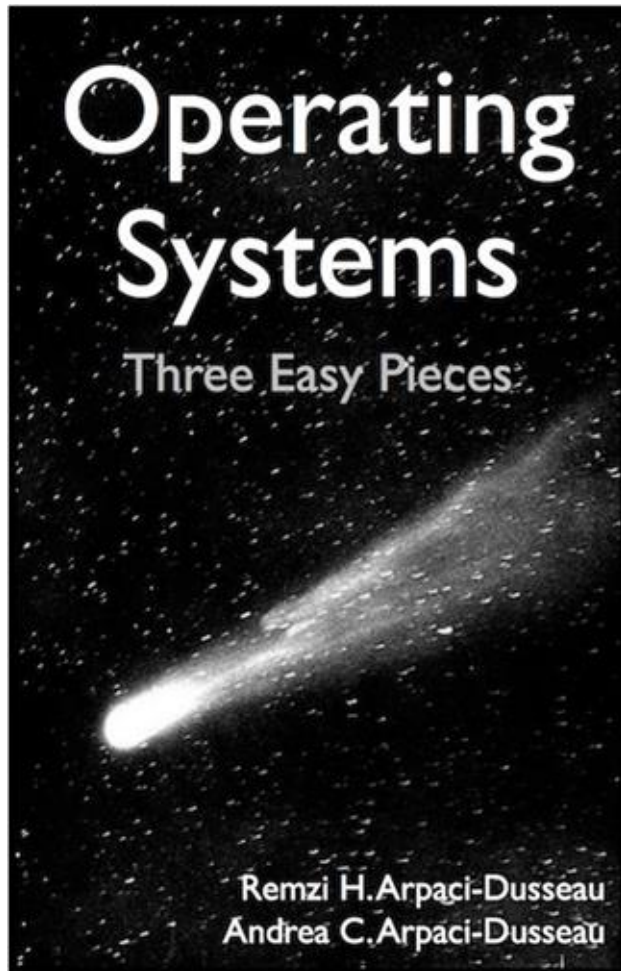
- Homework will consist of programming projects and questions from class
- Each project will be done individually
- Homework assignments must be submitted at the latest when they are due.
- You may submit multiple times as long as it's before the due date.
- We have an automatic grader that will give you feedback on realtime.
- We will use Anubis. (you used this in CompArch)

Textbook 1

- Throughout the semester we will be studying the source code of a UNIX-like operating system, **xv6**
- The authors of xv6 have written a short (100 page) book about its implementation, which accompanies the source code. You ***SHOULD*** read it.
- This has been **posted** on NYU Brightspace in the **Resources** section

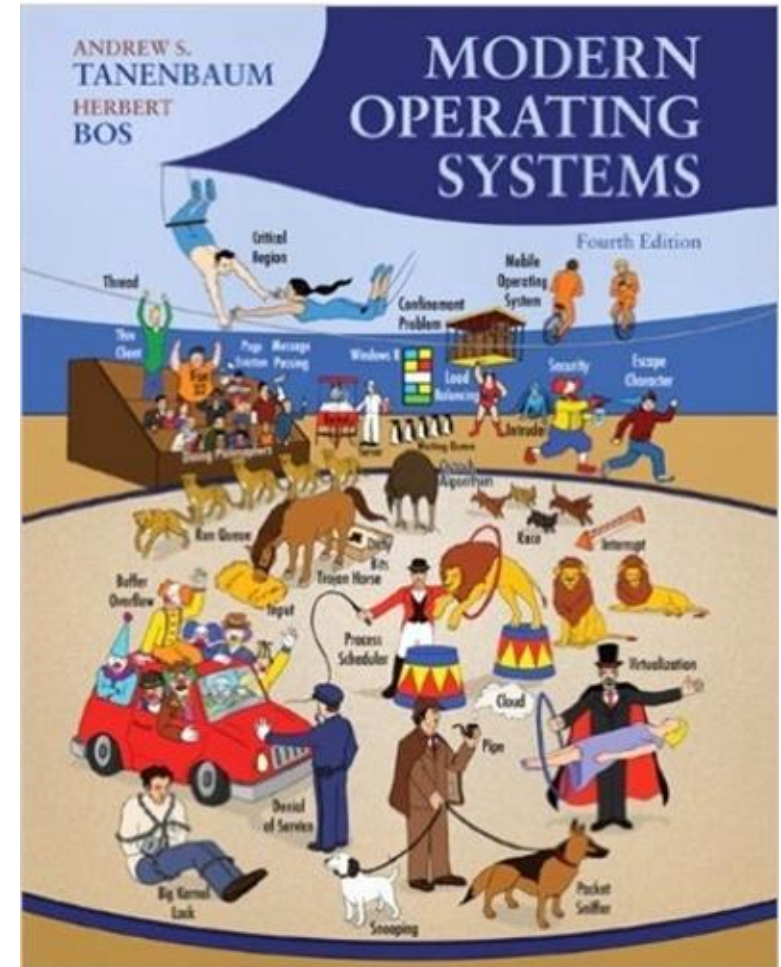
Textbook 2

- Operating Systems: Three Easy Pieces
- **Free online** at
<http://pages.cs.wisc.edu/~remzi/OSTEP/>
- Link in Brightspace



Textbook 3

- To complement lectures We will be using *Modern Operating Systems*, by Andrew S. Tanenbaum
- Ebook/PDF and international editions of the book are also fine.
- A PDF Copy has been **posted** on NYU Classes



Programming Language

- The homework will require you to write code in C
- If you have worked with C++; C is in most respects a subset of C++
- However, you may find that you need to understand the language in greater depth than you have before
- We will **review** C during class and a recitation
- If you have never programmed or never coded in C you **will have trouble** with this class and you might **FAIL**.

Grading

- **20%** Homework (Programming Projects)
- **35%** Midterm
- **35%** Final
- **10%** Participation (Polls 80% - >100%)

Optional recitation

- To help with Homework/Review
- Will depend on TA schedules
- Will be Taped
- Over zoom

Late days

- You get 24 hrs per assignment and then not accepted.

Syllabus

- In NYU Classes



A photograph of a New York University street scene, featuring a white building with a 'COLLEGE OF ARTS AND SCIENCE' sign and a blue NYU flag. The image is overlaid with a purple gradient.

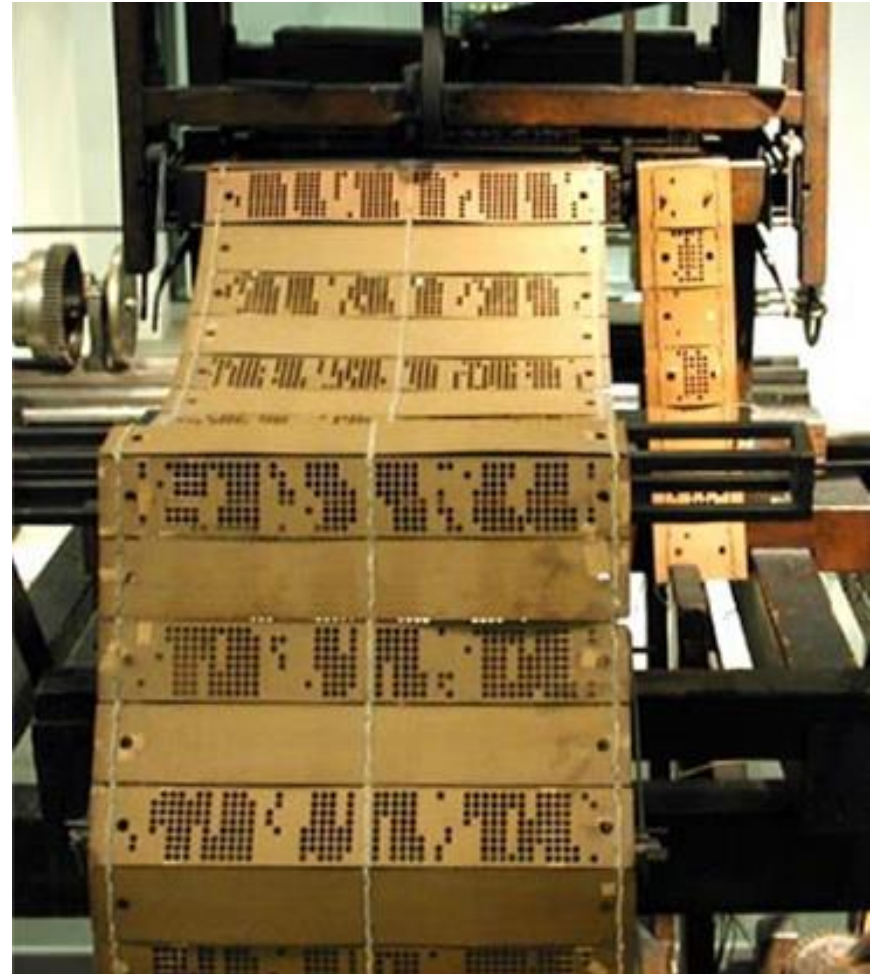
“

(Quick) Computer history

—

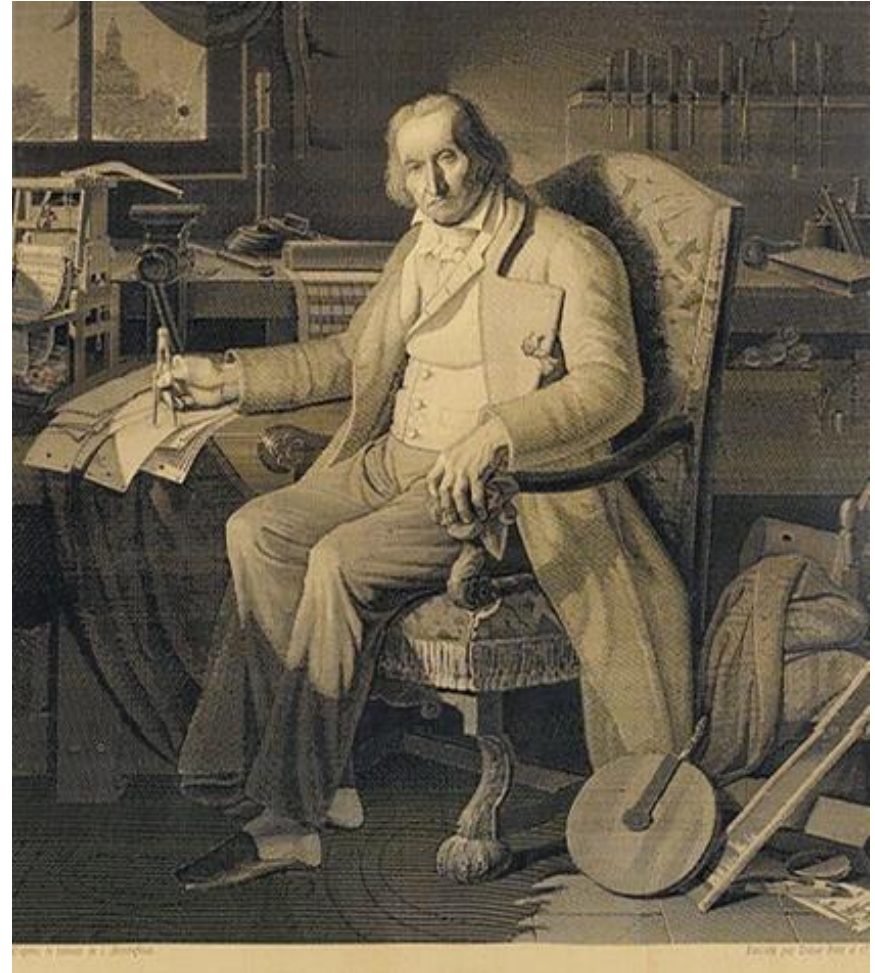
Pre-Pre-History

- The Jacquard Loom.
- Invented by Joseph Jacquard in 1801
- Used punched cards to describe patterns to weave
- We'll see such cards again...



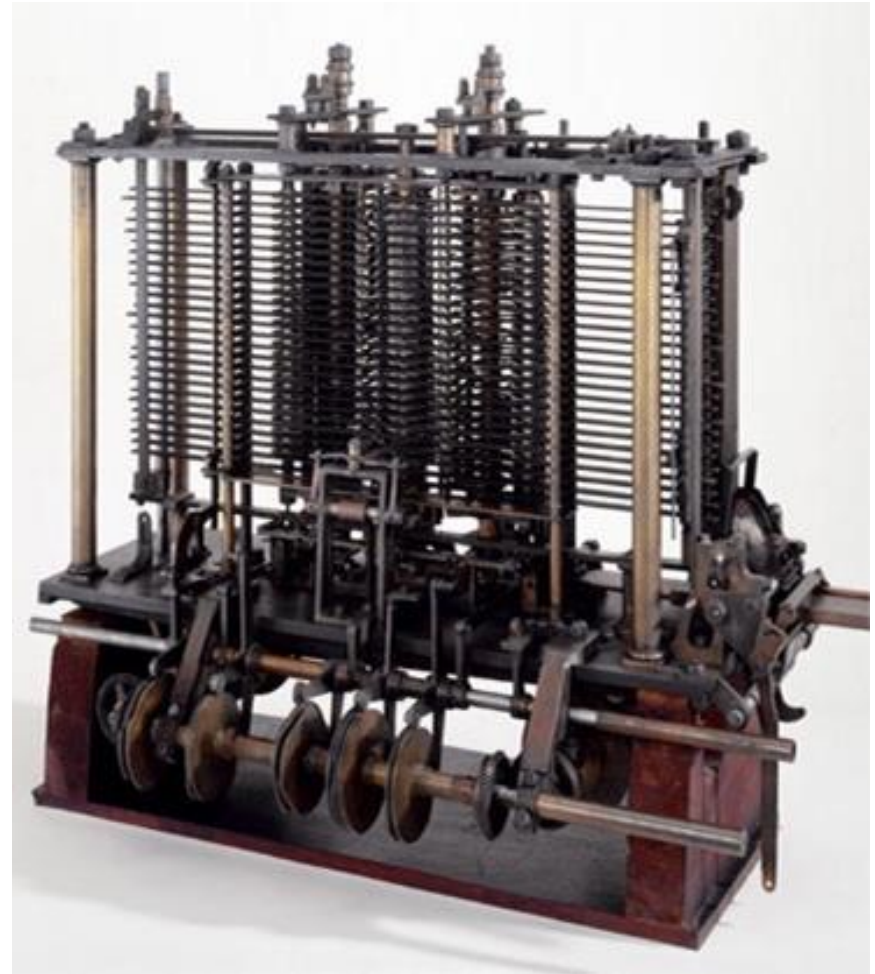
Pre-Pre-History

- Jacquard Loom could even be used to weave portraits
- This one took 24,000 punch cards to specify
- A picture is worth 1,000 words, or 24,000 punch cards



Pre-History

- The Analytical Engine: designed by Charles Babbage. (1837)
- Intended to be able to carry out arbitrary computations
- Never completed (lack of funding)
- Contained:
 - Arithmetic unit
 - Conditional Branching
 - Loops
- First “Turing Complete”



Prehistoric Programming

- [Ada Lovelace](#) (1815-1852) worked with Babbage to create programs for the Analytical Engine
- Created an algorithm to compute Bernoulli numbers on the machine
- First Programmer and also the first programmer to stay up all night working on code!



Z3 Machine (1941)

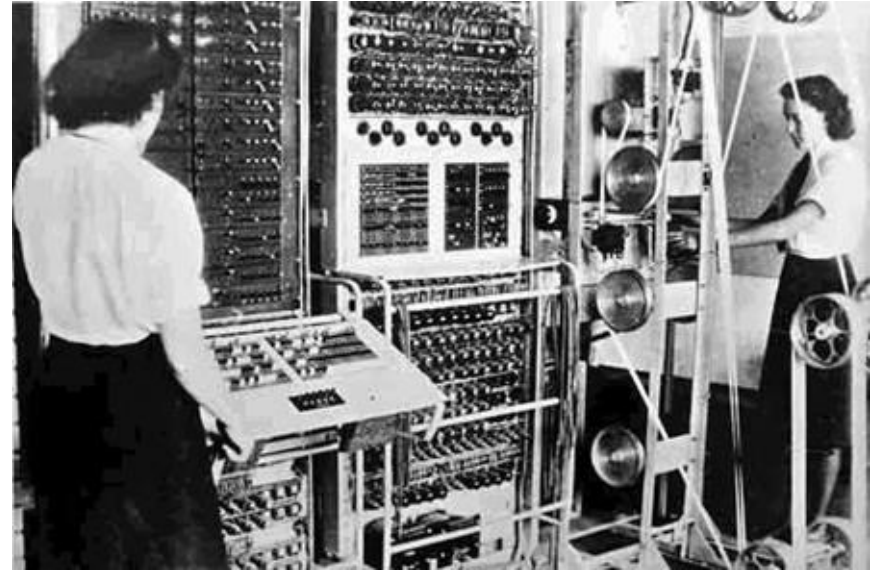
- Electromechanical relay computer built in Germany by Konrad Zuse
- Technically Turing-complete (proved in 1998), but typically just carried out specific calculations



Photo by Wikipedia User Jahoe

Colossus (1943)

- Designed by Tommy Flowers at Bletchley Park (England) for use in decrypting German radio teleprinter messages (not Enigma)
- One of the programmers was [Alan Turing](#)



Harvard Mark I (1944)

- Used for atomic bomb computations in Manhattan Project
- Program “loops” by physically connecting the end of the program tape to the start, forming a loop
- One of it’s first programmers was [Grace Hopper](#)

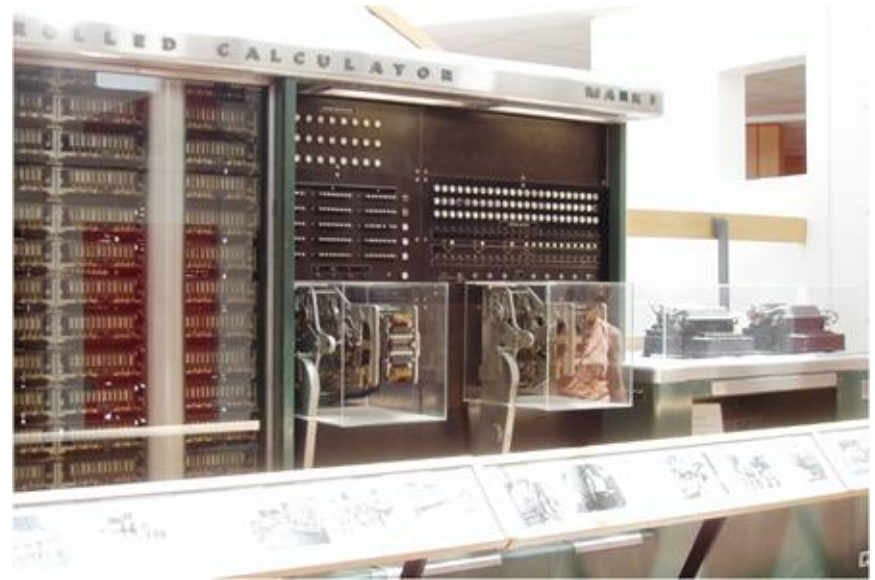


Photo by Wikipedia User Topory

ENIAC (1946)

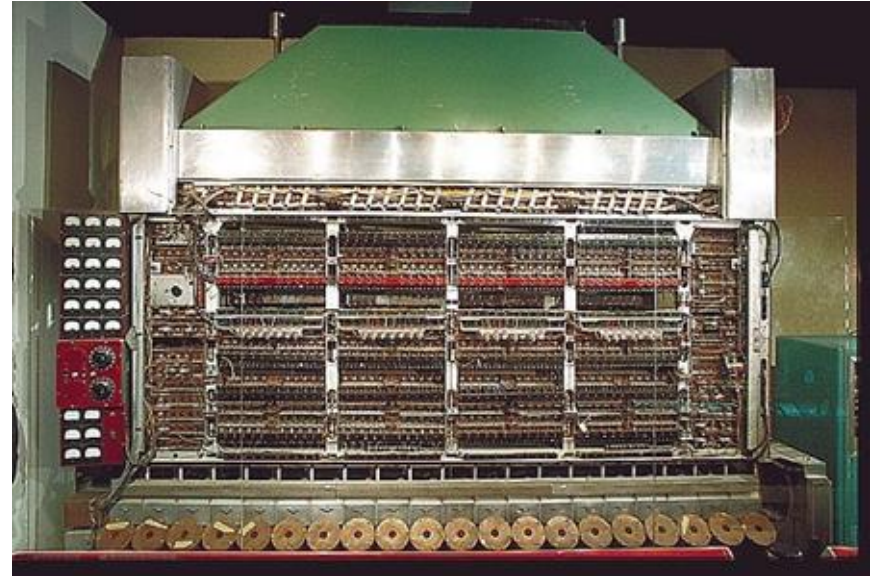
- Built at U Penn; used for artillery and hydrogen bomb computations



Photo by Wikipedia User TexasDex

Institute for advanced study (IAS) Machine (1951)

- One of the earliest to store code and data in the same memory
- Origin of the term “von Neumann Bottleneck”



Early Computing History

- Early machines were programmed directly in machine language (or even direct wiring!)
 - No C, Java, Python, etc.
 - Not even any assembly language

“Programs” on the IBM 402

- Each program is actually a plugboard wired up to configure the machine
- This picture is from a company called Sparkler Filters, which still uses its IBM 402 from 1948

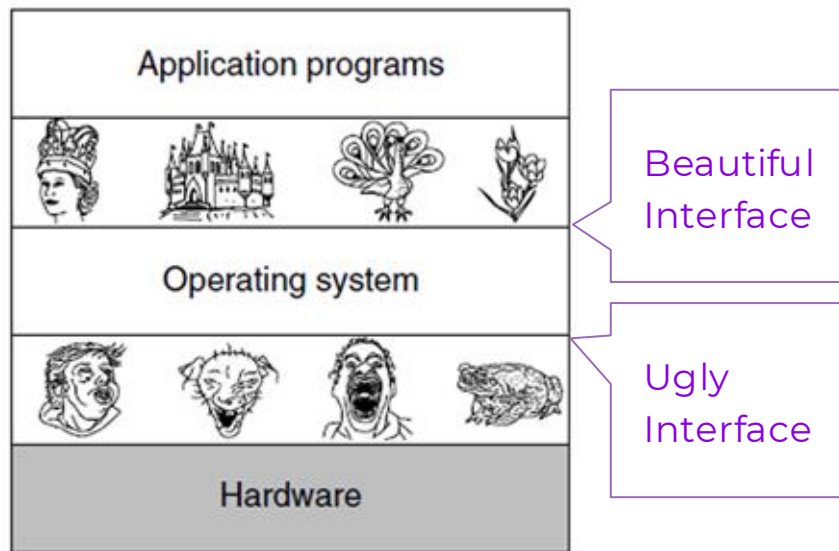




NYU

What is an Operating system ?

The Operating System as an Extended Machine



Operating systems turn ugly hardware into beautiful abstractions.

The Operating System as a Resource Manager

- **Top down view**
 - Provide abstractions to application programs
- **Bottom up view**
 - Manage pieces of complex system
- **Alternative view**
 - Provide orderly, controlled allocation of resources



History of operating systems

1. Vacuum Tube Technology

- Vacuum tubes functioned as high-speed electronic switches
- Access times for these machines are around 100 milliseconds (10⁻³) (1/10 of a second)



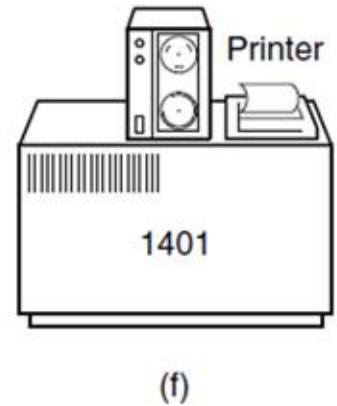
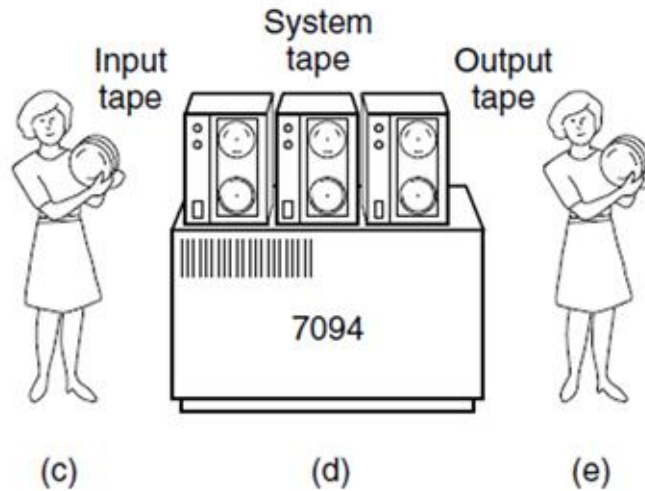
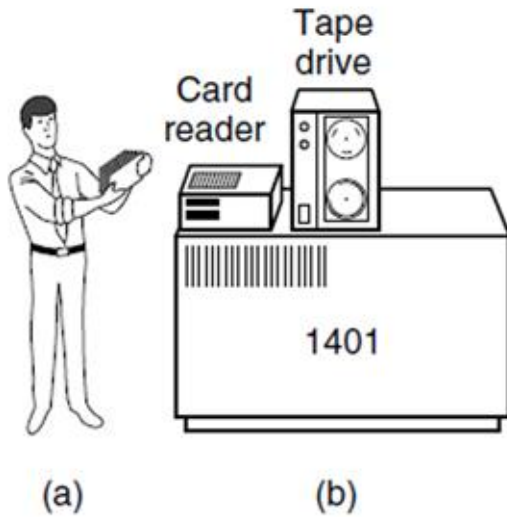
2. Transistor & Batch (1955-1965)

- Transistors allowed computers to be reliable enough for commercialization
- Computing time was typically metered
- Being able to utilize the CPU for the allotted time was very important.
- Transistor-based machines were also much faster
- Access times around $100\mu\text{s}$ – that's microseconds (10^{-6})
- This is a speedup of $\sim 1000\times$!

Batch Jobs

- An individual job took a set of punch cards
- Cards were loaded onto magnetic tape (using a fast reader), carried over to the actual computer, and then the output tape was brought over to a printer
- The key idea of batch processing is to collect multiple jobs which can be loaded onto the tape and then processed all at once

Batch jobs



3. ICs and multiprogramming (1965 – 1980)

- Integrated Circuits that contain many transistors
- Consolidated both the cheap and expensive machines
- IBM system/360 first one to use Integrated Circuits
- Multiprogramming
- Timesharing
- Started here MULTICS -> UNIX (FreeBSD, Linux, iOS, Android)

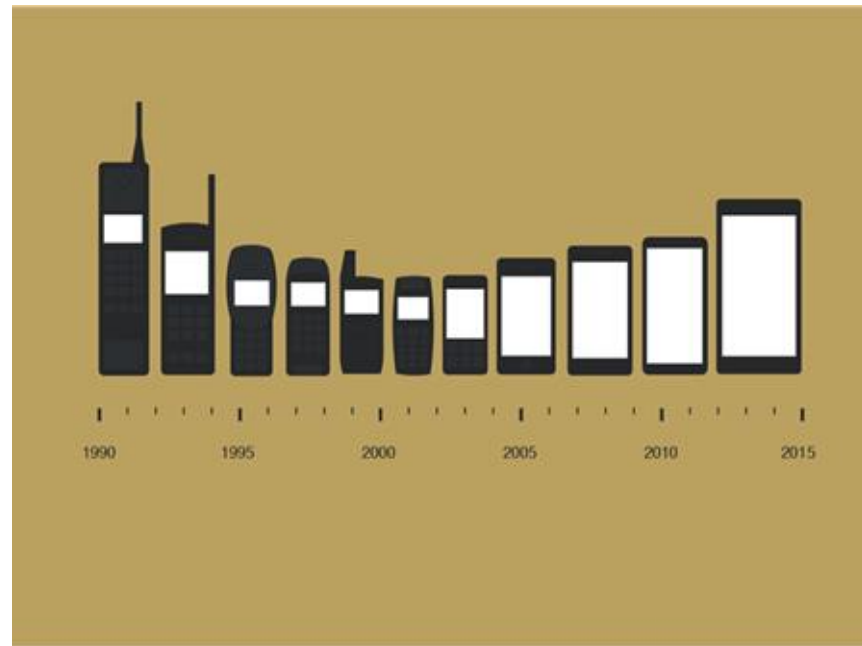
Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

4. Personal computers: (1980 – Present)

- Large Scale Integration Circuits (Chips) containing 1000s of transistor in a sq centimeter
- IBM 8080
- Microsoft DOS, Windows
- Apple

5. Mobile computers (1990 – Present)

- Nokia N9000/Symbian
- Windows Phone
- Blackberry
- iPhone/ iOS
- Android



6. ?? Your home??

- Alexa?
- Agents?

Class 1: Which OS are you using Today?

Windows

0%

Mac

0%

Linux

0%

None of the above

0%



Intro to operating systems

Operating System: Three Easy Pieces

What happens on the CPU when a program runs?

- **The processor:**
 - **Fetches** an instruction from memory
 - **Decodes** the instruction
 - **Executes** the instruction
 - Moves on to the next instruction

What is an Operating System?

Not easy to define precisely...



Operating System (OS):

Software that converts hardware into a useful **form** for applications

Operating system

Responsible For:

- Making it easy to run programs
- Allowing programs to share memory
- Enabling programs to interact with devices

OS is in charge of making sure the system operates correctly and efficiently.

Virtualization

- The OS takes a **physical resource** and transforms it into a **virtual form** of itself.
 - **Physical Resource:** Processor, Memory, Disk ...
 - The Virtual form is more general, powerful, and easy-to-use
 - Sometimes, we refer to the OS as a **virtual machine**.

OS as resource manager

- The OS manages resources such as CPU, memory and disk
- The OS allows
 - Many programs to run -> sharing CPU
 - Many programs to concurrently access their own instructions and data => sharing memory
 - Many programs to access devices => sharing disks

Three PIECES: FIRST

- **Virtualization**
 - Make each application believe it has each resource to itself

Virtualizing Memory (cont.)

- **Each process accesses its own private virtual address space.**
 - The OS maps **address space** onto the **physical memory**.
 - A memory reference within one running program **does not affect** the address space of other processes.
 - Physical memory is a **shared resource**, managed by the OS.

Three PIECES: SECOND

- **Concurrency**

Events are occurring simultaneously and may interact with one another

- OS must be able to handle concurrent events
- Easier case
 - Hide concurrency from independent processes
- Trickier case
 - Manage concurrency with interacting processes
 - Provide abstractions (locks, semaphores, condition variables, shared memory, critical sections) to processes
 - Ensure processes do not deadlock

Three PIECES: Third

- **Persistence:** Access information permanently
 - Lifetime of information is longer than lifetime of any one process
 - Machine may be rebooted, machine may lose power or crash unexpectedly
 - **Share** information
- Issues:
 - Provide abstraction so applications do not know how data is stored : Files, directories (folders), links
 - **Correctness** with unexpected failures
 - **Performance:** disks are very slow; many optimizations needed!
- Demo
 - File system does work to ensure data updated correctly

Persistence

- What the OS does to write to disk?
 - Figure out **where** on disk the data will reside
 - **Issue I/O** requests to the underlying storage device
- File system handles system crashes during write
 - **Journaling** or **copy-on-write**
 - Carefully ordering writes to disk

Advanced Topics

- Current systems
 - Multiprocessors
 - Networked and distributed systems
 - Virtual machines

OS Design goals

- Build up some **abstractions**
 - Make the system convenient and easy to use
- Provide **High Performance**
 - Minimize the overhead of the OS
 - OS must strive to provide virtualization without excessive overhead
- **Protection** between applications
 - Isolation: Bad behavior of one does not harm other and the OS itself

OS Design goals (cont.)

- High degree of reliability
 - The OS must also run non-stop
- Other issues:
 - Energy- efficiency
 - Security
 - Mobility

Why study Operating Systems?

Build, modify, or administer an operating system to understand system performance

- Behavior of OS impacts entire machine
- Tune workload performance
- Apply knowledge across many layers
 - Computer architecture, programming languages, data structures and algorithms, and performance modeling

Fun and challenging to understand large, complex systems